StatSpeak Architecture & Guide

The Student Feedback Quantifier - Complete Project, Architecture & Code Guide

1. Project Overview & Objectives

StatSpeak is a professional, standalone desktop application built entirely on Java 17 and Core Java technologies. It was engineered to analyze qualitative student feedback, identify high-frequency keywords, extract critical educational topics, and generate statistical summaries and visual distributions. A foundational mandate of StatSpeak is 100% local, offline execution to guarantee student privacy without transmitting sensitive academic evaluations to external cloud APIs.

2. Modular System Architecture

The project is structured into distinct, decoupled packages following Model-View-Controller (MVC) principles:

* **com.statspeak.app:** Contains Main.java, the bootstrap entry point configuring FlatLaf modern UI and launching the event dispatch thread.
* **com.statspeak.model:** Immutable data carriers: FeedbackEntry (raw records) and AnalysisResult (aggregated keyword statistics).
* **com.statspeak.importdata:** Robust CSV parser handling multi-row fields, quotes, and malformed records.
* **com.statspeak.analysis:** TextProcessor performing normalization, punctuation stripping, configurable stop-word filtering, and HashMap frequency calculations.
* **com.statspeak.visualization:** ChartRenderer generating anti-aliased Java2D gradient horizontal bar charts.
* **com.statspeak.ui:** StatisticDashboard providing the Swing GUI, KPI cards, JTable views, and responsive tabbed panels.
* **com.statspeak.reports:** ReportsExporter and PdfReportExporter producing CSV, plain text, and Apache PDFBox PDF reports.

3. Internal Pipeline & Execution Flow

When an educator analyzes a dataset, the application executes a deterministic 5-stage pipeline:

* **Stage 1 - Ingestion & Validation:** CSVDataImporter streams the file with BufferedReader. It validates headers, identifies quotes, handles empty lines, and constructs a List of FeedbackEntry objects.
* **Stage 2 - Text Normalization:** TextProcessor converts characters to lower case, removes non-alphanumeric punctuation using regular expressions, and tokenizes into words.
* **Stage 3 - Stop-Word Elimination:** Configured common words (the, is, at, which, on, etc.) are purged using an in-memory HashSet for O(1) lookups.
* **Stage 4 - Frequency Aggregation:** Words are inserted into a HashMap<String, Integer>. The occurrences are sorted via an ArrayList and custom Comparator implementing deterministic frequency ranking.
* **Stage 5 - Presentation & Export:** Results populate the GUI KPI dashboard, Java2D chart render buffer, and export handlers for CSV/PDF generation.

4. Technology Stack & Modern UI

StatSpeak combines enterprise Java standards with modern styling:

* **Java 17 LTS:** Leveraging modern language features, record-style immutability, and robust performance.
* **FlatLaf 3.4.1:** Eliminated legacy bevels in favor of modern 12px rounded components, Slate & Indigo palettes, and 36px table row breathing room.
* **Apache PDFBox 2.0.29:** Local cryptographic PDF rendering for academic report exports.
* **JUnit 5:** Automated unit and integration test coverage across all core modules.
* **Gradle 8.5:** Automated wrapper build and standalone Fat-JAR packaging.

5. How to Run and Share

1. Direct Execution:
   - Double click 'StatSpeak.jar' located in the project root (requires Java 17+ installed).
   - Or run in terminal: .\gradlew.bat run

2. Sharing with Colleagues:
   - Send 'StatSpeak-Release.zip' (created on Desktop, ~5MB).
   - Recipient unzips and runs without needing build tools or Gradle pre-installed.